

Verzioniranje i kolaborativno programiranje s gitom

Praktikum: Razvojne operacije · Tjedan 2 · Blok A

Sadržaj bloka

1	**Verzioniranje izvornog koda**	Tko, kada i zašto
2	**Strategije spajanja promjena**	Fast-forward, merge commit, squash, rebase
3	**Strategije razvoja na GitHubu**	Grananje, okruženje = grana, Actions, tajne, zaštita, RBAC
4	**Vježba: zajednički repozitorij**	Grana, skripta, pull request prema main
5	**Provjera razumijevanja**	5 kratkih pitanja

1

Verzioniranje izvornog koda

Tko, kada i zašto

Tko, kada i zašto?

Sustav za verzioniranje pamti svaku promjenu koda i odgovara na tri pitanja:

T Tko?

Autor svake promjene: ime i e-mail iz git configa (tjedan 1).

```
git blame app.py
```

K Kada?

Točan trenutak svake promjene. Povratak na bilo koje stanje u prošlosti.

```
git log --since=2.weeks
```

Z Zašto?

Poruka commita i poveznica na issue ili pull request objašnjavaju namjeru.

```
git show a1b2c3d
```

Zašto uopće? Sigurna kopija, rad više ljudi na istom kodu bez prepisivanja, revizija (audit) i automatizacija: CI se pokreće na svaki commit.

Od lokalnog do distribuiranog verzioniranja

1 — 2 — 3 — 4

Local

RCS (1982)

Povijest samo na jednom računalu

Centralized

CVS, SVN (2000)

Jedan poslužitelj, bez mreže nema commita

Distributed

git (2005)

Svaki klon ima cijelu povijest

Platforms

GitHub (2008)

Pull requestovi, review, Actions

git je napisao Linus Torvalds 2005. za razvoj Linux kernela: tisuće programera, bez središnjeg poslužitelja, brzo grananje. GitHub je platforma oko gita, a ne git sam.

Četiri mjesta na kojima kod živi

Working directory

datoteke koje uređujemo

Staging area

što ide u sljedeći commit

Local repository

povijest na našem računalu (.git)

Remote (GitHub)

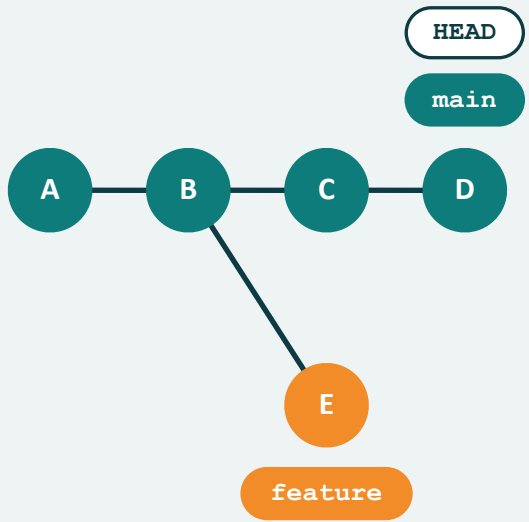
zajednička povijest tima




```
git status              # what changed, what is staged
git diff --staged       # exactly what the next commit contains
git restore --staged app.py   # take a file back out of staging
```

Commit, grana i HEAD

Commit graph



Strelice pokazuju na roditelja: svaki commit zna od čega je nastao.

1 Commit

Snimka cijelog projekta + autor, datum, poruka i roditelj. Ime je hash, npr. a1b2c3d.

2 Grana (branch)

Pomični pokazivač na jedan commit. Novi commit pomiče granu naprijed.

3 HEAD

Gdje smo sada: grana na kojoj radimo. git checkout premješta HEAD.

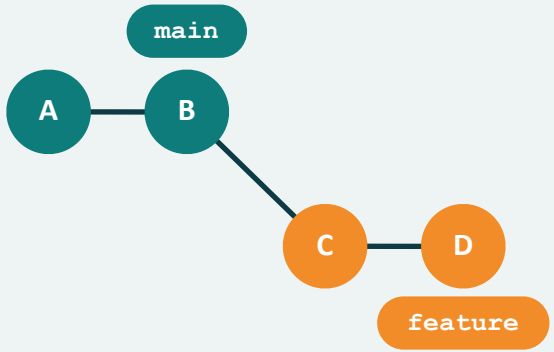
2

Strategije spajanja promjena

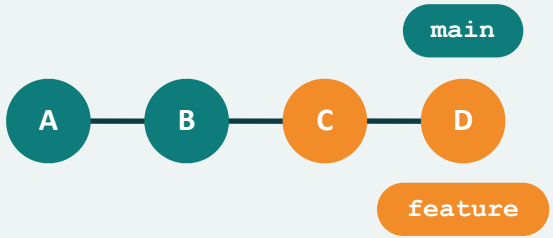
Fast-forward, merge commit, squash, rebase

Fast-forward

Before



After



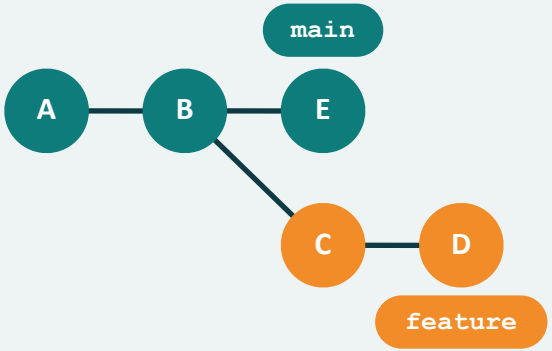

```
git checkout main
git merge --ff-only feature

# every day: git pull
# "Updating 1a2b..9f8e Fast-forward"
```

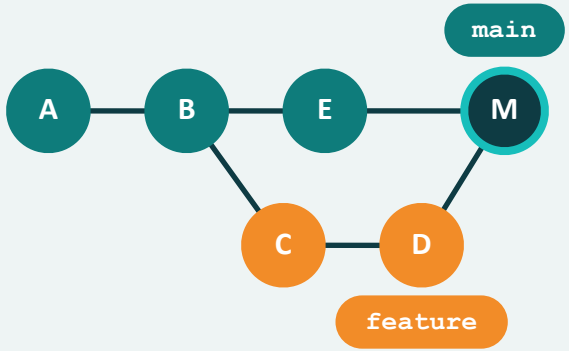
- main se samo pomakne naprijed na D
- Nema novog commita, povijest ostaje ravna
- Moguće samo ako main nije napredovao
- git pull to radi gotovo svaki put

Merge commit (3-way merge)

Before



After

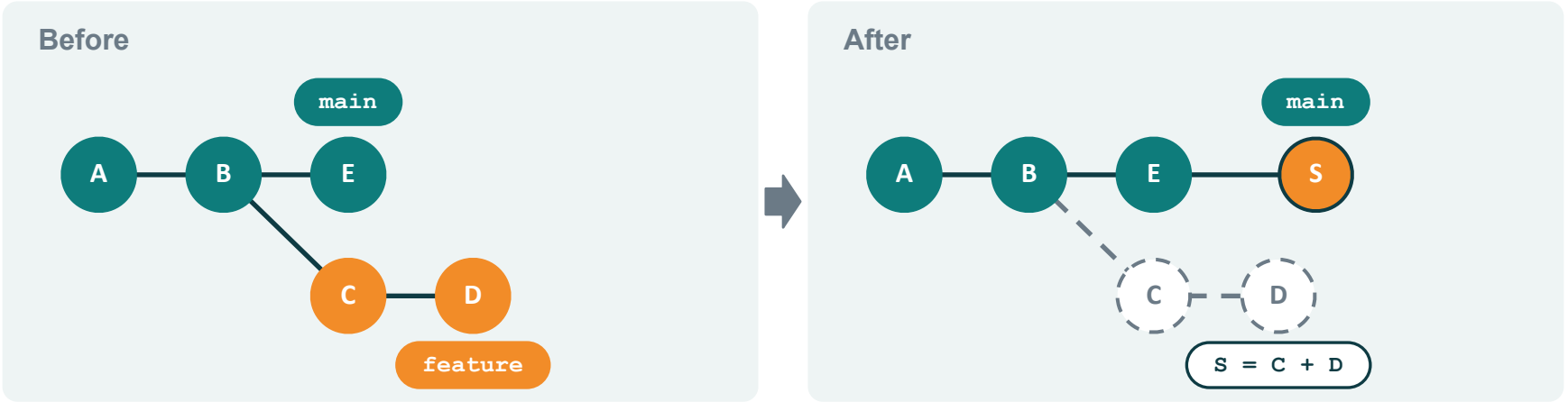



```
git checkout main
git merge --no-ff feature

# GitHub: "Create a merge commit"
```

- Kolega je u međuvremenu dodao E na main
- Novi commit M ima dva roditelja: E i D
- Čuva svaki commit i trenutak spajanja
- Povijest postaje razgranata

Squash



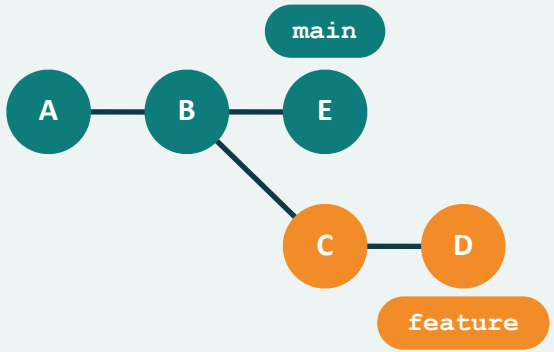

```
git checkout main
git merge --squash feature
git commit -m "feat: add login form"

# GitHub: "Squash and merge"
```

- C i D se sažmu u jedan novi commit S
- Jedan pull request = jedan commit na main
- Gube se pojedinačni koraci s grane
- Naslov PR-a postaje poruka (Conventional Commits)

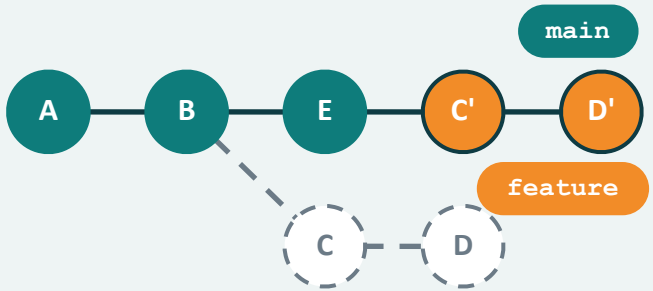
Rebase

Before



After

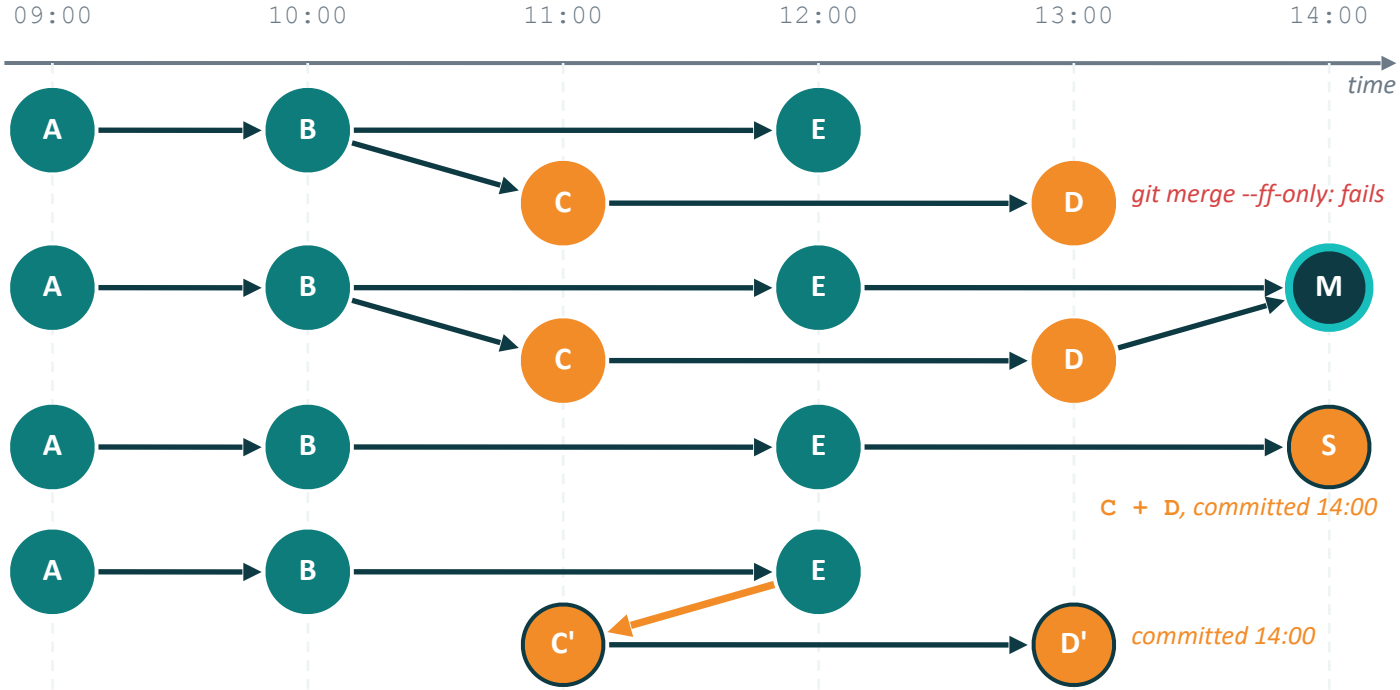
vs fast-forward: commits are copied, not just pointed to




```
git checkout feature
git rebase main   # replay C, D on E
git checkout main
git merge --ff-only feature
# GitHub: "Rebase and merge"
```

- C i D se ponovno odigraju na vrhu E
- Novi commitovi C' i D' imaju nove hasheve
- Razlika od FF-a: FF samo pomakne main, rebase prvo stvara nove commitove
- Ne rebaseati grane koje drugi već koriste

Vremenska komponenta spajanja



Fast-forward

Nije moguć dok main ima E: tada merge commit.

Merge commit

Izvorna vremena + M u trenutku spajanja.

Squash

Rad dobiva vrijeme spajanja.

Rebase

Na main: A→B→E→C'→D'
C' (11:00) dolazi iza E (12:00).

Položaj kruga = author date (kada je promjena napisana). Strelica vodi od roditelja prema djetetu: tako se čita redoslijed na main.

Usporedba strategija

Strategija	Novi commit	Povijest na main	Hashevi grane	Gumb na GitHubu
Fast-forward	ne	ravna	ostaju isti	nema (najbliže: Rebase and merge)
Merge commit	da, M s 2 roditelja	razgranata, potpuna	ostaju isti	Create a merge commit
Squash	da, jedan S	ravna, 1 commit po PR-u	ne završe na main	Squash and merge
Rebase	da, kopije C', D'	ravna, svi commitovi	novi hashevi	Rebase and merge

Kako odabrati?

Tim bira jednu strategiju i zapiše je u postavke repozitorija. Na ovom kolegiju: fast-forward kad je moguć, inače merge commit. To je zadano ponašanje naredbe git merge.

Zlatno pravilo

Ne prepisuj povijest koju su drugi već povukli: bez rebasea i force pusha na main.

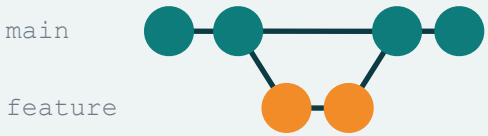
3

Strategije razvoja na GitHubu

Grananje, okruženja, Actions, tajne, zaštita grana i ovlasti

Strategije grananja

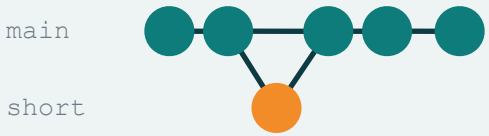
GitHub Flow



- main je uvijek spreman za isporuku
- Grana → PR → review → spajanje
- Isporuka nakon svakog spajanja

Kada: Mali timovi, web aplikacije. Ovo koristimo na kolegiju.

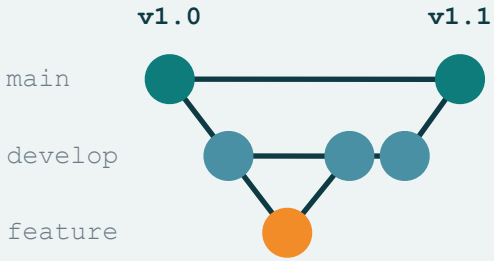
Trunk-based



- Integracija u main barem dnevno
- Grane žive satima, ne danima
- Nedovršeno skriveno feature flagom

Kada: Zreo CI s puno automatskih testova.

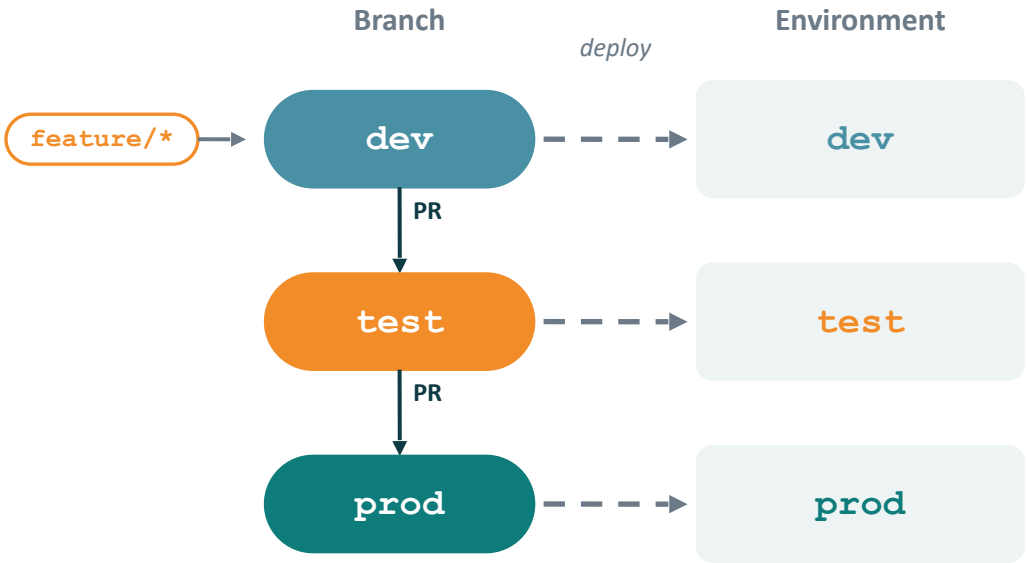
Gitflow



- Trajne grane: main i develop
- Plus feature/*, release/*, hotfix/*
- Izdanja po planu, s tagovima

Kada: Rijetka izdanja: mobilne i desktop aplikacije, biblioteke.

Konvencija: ime okruženja = ime grane



U vježbi danas koristimo samo main (GitHub Flow). Okruženja detaljno u temi 11.

Zašto ista imena?

- Jedan workflow za sva okruženja: environment se čita iz imena grane.
- Iste varijable i tajne (API_URL, API_KEY), druge vrijednosti po okruženju.
- Promocija = pull request iz niže grane u višu.
- Svatko iz imena grane zna gdje će kod završiti.

GitHub Actions, deploymenti i tajne

```yaml
name: deploy

on:
  push:
    branches:
      - dev
      - test
      - prod

jobs:
  deploy:
    runs-on: ubuntu-latest
    environment: ${{ github.ref_name }}
    steps:
      - uses: actions/checkout@v4
      - name: Deploy
        env:
          API_URL: ${{ vars.API_URL }}
          API_KEY: ${{ secrets.API_KEY }}
        run: ./deploy.sh "$API_URL"
```

Environment

Ime okruženja = ime grane. Svaki run je zapisan kao deployment, s pravilima (npr. obavezno odobrenje za prod).

vars.*

Konfiguracija koja nije tajna: URL-ovi, imena modela. Vidljiva u logu.

secrets.*

Šifrirane vrijednosti, maskirane u logu (***). Nikad u kodu ni u .env u repozitoriju.

```
gh variable set API_URL --env dev
gh secret set API_KEY --env dev
```

Zaštita grana (branch protection)

Pravila koja GitHub provodi na grani main, bez obzira na to tko pusha:

1 Require a pull request

Nema izravnog pusha na main; sve ide kroz PR.

2 Require approvals

Barem jedan pregled (review) prije spajanja.

3 Require status checks

CI mora biti zelen (testovi, lint).

4 Require linear history

Dopušteni samo squash i rebase.

5 Block force pushes

Nitko ne može prepisati povijest main grane.

6 Restrict deletions

Grana main ne može se obrisati.

Gdje: Settings → Rules → Rulesets (noviji način) ili Settings → Branches. Pravila za cijelu organizaciju postavlja vlasnik organizacije.

RBAC: tko smije što

Ovlast	Read	Triage	Write	Maintain	Admin
Čitanje i kloniranje	✓	✓	✓	✓	✓
Issues i oznake	–	✓	✓	✓	✓
Push grana, spajanje PR-a	–	–	✓	✓	✓
Postavke repozitorija	–	–	–	dio	✓
Zaštita grana i tajne	–	–	–	–	✓

Teams

Ovlasti dajemo timu u organizaciji, ne pojedincu.

CODEOWNERS

Datoteka koja određuje tko mora odobriti promjene u kojem direktoriju.

Environment reviewers

Tko smije odobriti isporuku u prod.

Naš zajednički repozitorij: tim studenti ima Write (push grana, otvaranje PR-a), nositelj ima Admin. Zaštita grane main znači da ni Write ne smije pushati izravno na main.

4

Vježba: zajednički repozitorij

Grana, skripta, pull request prema main

Tijek vježbe

1 — Clone

```
gh repo clone
<org>/shared-collab
```

2 — Branch

```
git checkout -b
feature/<login>
```

3 — Script

```
bash
add_contribution.sh
```

4 — Push

```
git push -u
origin HEAD
```

5 — Pull request

```
gh pr create
--base main
```

6 — Merge

```
nositelj spaja
na 3 načina
```

Što radi skripta

- Datoteka 2026-10-20T10:15:42.txt (Europe/Zagreb)
- U njoj je korisničko ime ($USER)
- Commit datoteke + jedan prazan commit

Zašto dva commita?

Svaki PR ima dva commita, pa se pri spajanju jasno vidi razlika: squash ih sažme u jedan, rebase ih kopira oba, merge commit doda treći.

Skripta add_contribution.sh

```
# refuse to run on main: work on your own branch
branch=$(git branch --show-current)
[ "$branch" = "main" ] && exit 1

# ISO 8601, local Zagreb time, no ms, no TZ
timestamp=$(TZ=Europe/Zagreb date +%Y-%m-%dT%H:%M:%S)
file="${timestamp}.txt"
echo "$USER" > "$file"

git add "$file"
git commit -m "feat: add contribution from $USER"
git commit --allow-empty \
  -m "chore: empty commit from $USER"
```

`TZ=Europe/Zagreb`

Isto vrijeme za sve, bez obzira na postavke računala.

`--allow-empty`

Commit bez promjena datoteka. Koristan za pokretanje CI-a ili oznake u povijesti.

`feat: / chore:`

Conventional Commits, kao u release-please primjeru iz tjedna 1.

Windows: ":" nije dopušten u imenu datoteke. Raditi u WSL-u.

Spajanje uživo: tri PR-a, tri načina

PR 1

Rebase and merge

2 nova commita na main, novi hashevi

PR 2

Squash and merge

1 novi commit na main

PR 3

Create a merge commit

2 commita s grane + merge commit

```
git checkout main
git pull                    # Fast-forward
git log --graph --oneline --all

# behind main? update your own branch:
git fetch origin
git rebase origin/main
git push --force-with-lease
```

Pokušajte i ovo

`git push origin main`

Zaštita grane odbija push: "protected branch hook declined".

Provjera razumijevanja

1. Na koja tri pitanja odgovara povijest u gitu?
2. Kada je fast-forward moguć, a kada nije?
3. Što se dogodi s commitovima C i D nakon squasha, a što nakon rebasea?
4. Zašto ne smijemo rebaseati granu main?
5. Koja dva pravila zaštite grane sprječavaju izravan push nepregledanog koda?